

TimeAudit 个人工时数仓遥测系统： Windows PowerShell 高可靠性快速部署与 灾备指南

本指南专为 **TimeAudit（个人工时数仓遥测系统）** 量身定制。基于对系统底层集成控制源码（`start_all.bat`、`docker-compose.yml`、`Dockerfile`、`check_status_gui.ps1` 等）的最新技术推演与物理目录映像分析，本指南旨在指导如何在 Windows 11 宿主机环境下，利用 PowerShell (PS) 实现全套高精遥测探针、时序持久化层 (PostgreSQL 15)、异步清洗管道 (Ingestor) 及可视化大屏 (Grafana) 的无缝并网、高权限自启与灾后无损重建。

系统时空对齐与自启引导机理

为了确保数据流在长周期追踪中不发生断流或时序错位，TimeAudit 在宿主机与容器群之间建立了一套精密的技术协同防线：

- 绝对时空对齐机制：**整个数仓的指标写入与计算均以 UTC 统一时间轴为基准。为了终结 Windows 宿主机与 Linux 容器之间常见的 8 小时时区漂移虚警，实况体检程序直接穿透至 PostgreSQL 容器内部，通过 `SELECT EXTRACT(EPOCH FROM (now() - timestamp)) FROM public.fact_system_hardware ORDER BY timestamp DESC LIMIT 1`；动态计算秒级物理时差。当且仅当该物理时差小于 10.0 秒时，系统才宣告绿色健康。
- 链式提权引导架构：**整个系统的自启完全收敛于 `start_all.bat`。脚本首先将控制台代码页强制切换为 UTF-8（`chcp 65001 >nul`）以放行内嵌 Emoji 与中文渲染，随后拉起 `TimeAudit.ahk` 状态机内核，利用 `netstat -ano | findstr 55432` 循环点检 **55432** 端口直到数据库物理就绪，最终通过 PowerShell 穿透 UAC 边界，以完全隐形的无窗口模式（`-windowStyle Hidden`）与管理员特权（`-verb RunAs`）分派 Python 遥测核心进程 `main.py`。

基础设施点检与网络代理整備

在执行任何自动化脚本前，请确保项目文件已严格按照物理拓扑部署在 `E:\TimeAudit` 路径下。

1. 物理目录结构资产核对

根据最新的系统物理快照，您的工作目录已完整集成了所有必需的原生驱动、维度卡片及配置资产。请严格点检以下资产矩阵：

资产分类	物理文件/夹名称	核心技术职能
自启与监控引擎	<code>start_all.bat</code> <code>TimeAudit.ahk</code> <code>check_status_gui.ps1</code>	负责整套管线的无窗提权引导、AHK 状态机内核初启、以及生成 UTF-16 编码的实况体检报告。
容器集群编排	<code>docker-compose.yml</code> <code>Dockerfile</code>	负责 PostgreSQL 时序库、Grafana 大屏及本地 Ingestor 容器环境的秒级并网。

资产分类	物理文件/夹名称	核心技术职能
宿主机遥测探针	<code>main.py</code> <code>context_worker.py</code> <code>hardware_worker.py</code> <code>activity_worker.py</code> <code>lifecycle_worker.py</code>	宿主机 Python 核心调度流。多轨并发采集前台聚焦窗口、网卡流量影子流、驱动级显卡（NVML）能耗及内存级进程生命周期。
外部二进制底层驱动	<code>LibreHardwareMonitor.exe</code> <code>PresentMonConsole.exe</code>	高精硬件遥测的物理底座。前者建立 WMI 命名空间（ <code>root\LibreHardwareMonitor</code> ）读取 CPU VCore 电压；后者拦截 3D 渲染帧率。
时序持久化挂载卷	<code>postgres_data/</code> <code>grafana_data/</code> <code>grafana_provisioning/</code>	物理机卷映射。存储全量事实表数据、大屏快照及脱水纯文本自动化卡片配置（Provisioning）。

2. 核心持久层凭据与端口矩阵

微服务群的通信边界与防线放行指标已硬编码如下，请勿在后续操作中随意篡改：

- **时序数仓 (PostgreSQL 15)**：宿主机映射暴露端口为 `55432`（容器内部物理端口 `5432`）。超级用户名：`leyang` | 密码：`SecurePassword123` | 默认数据库：`time_audit` | 持久化物理路径：`./postgres_data`。
- **可视化大屏 (Grafana OSS)**：宿主机映射暴露端口为 `53000`（容器内部物理端口 `3000`）。开机默认启用了浅色主题（Light Theme），并强制放行了 HTML 标签净化以保证高级卡片的无感渲染。

3. 国内网络代理与容器构建整備

由于 `docker-compose.yml` 在初次并网时需要动态读取本地 `Dockerfile` 并基于 `python:3.10-slim` 基础镜像执行 `apt-get` 编译，构建阶段需要拉取 Linux 原生驱动包 `libpq5` 与 `psycopg` 库，国内宿主机极易遇到网络握手断流或超时。

为了确保容器管道一键构建成功，部署前必须为 **Docker Desktop** 强制对齐本地代理环回网关：

1. 打开 Docker Desktop 图形界面，点击右上角齿轮图标进入 **Settings**。
2. 导航至左侧菜单栏的 **Resources** -> **Proxies** 选项卡。
3. 将 **Proxy mode** 明确切换为 **Manual configuration**。
4. 在 **Docker Desktop proxy**（用于拉取基础镜像）与 **Containers proxy**（用于容器出站流量）中，统一配置本地网关：
 - **Web Server (HTTP)**: `http://127.0.0.1:7892`
 - **Secure Web Server (HTTPS)**: `http://127.0.0.1:7892`
5. 点击右下角 **Apply & restart** 按钮，静待 Docker 守护进程重载网络。

🔧 第一步：启用虚拟化架构与安装 WSL 2 底座

TimeAudit 系统的关系型时序持久层与 Ingestion 数据管道完全寄宿于轻量级 Linux 容器中，而 Windows 侧的 Docker Desktop 必须依赖 WSL 2 (Windows Subsystem for Linux) 作为其原生 Hypervisor 拓扑底座。

请在 Windows 搜索栏输入 `PowerShell`，右键选择“以管理员身份运行”，依次复制并执行以下控制指令：

```
# 1. 开启 windows subsystem for linux 全局内核组件
Write-Host "[*] 正在启用 Linux 子系统底层组件..." -ForegroundColor Cyan
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux -NoRestart

# 2. 开启 虚拟机平台 组件（WSL 2 运行的核心虚拟化拓扑依赖）
Write-Host "[*] 正在并网虚拟机平台拓扑..." -ForegroundColor Cyan
Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -NoRestart

# 3. 将 WSL 内核强制升级至微软官方最新稳定版本
Write-Host "[*] 正在强制分发并更新 WSL 2 生产级内核..." -ForegroundColor Cyan
wsl --update

# 4. 验证 WSL 运行状态与底层架构
Write-Host "[+] WSL 底座第一阶段初始化完毕，当前版本状态：" -ForegroundColor Green
wsl --status
```

⚠️ **关键灾备提示：**执行完毕上述虚拟化组件安装后，**必须手动重启一次宿主机计算机**。只有在重启后，Windows 虚拟机管理程序（Hypervisor）才会正式开放物理端口转发与本地卷的动态挂载拓扑，否则后续的 Docker 启动命令将直接抛出底层文件系统锁死异常。

📦 第二步：利用 Winget 静默一键并网 Docker 与 Python 环境

为了规避传统图形化安装包因权限、多版本冲突及复杂的点击确认引发的配置漂移，本阶段全部采用 Windows 原生包管理器 `winget` 进行全自动、无感静默部署，确保宿主机环境与容器内运行环境（`python:3.10-slim`）保持高度技术对齐。

请在计算机重启后，重新打开一个 **管理员权限** 的 PowerShell 终端，复制并执行以下指令：

```
# 1. 自动化下载并静默安装 Docker Desktop 生产版（跳过一切协议弹窗）
Write-Host "[*] 正在通过基础设施管线静默分发 Docker Desktop..." -ForegroundColor Cyan
winget install Docker.DockerDesktop --silent --accept-package-agreements --accept-source-agreements

# 2. 全自动安装 Python 3.10 环境（与微服务 Dockerfile 基础镜像内核高度对齐，防止开发层语法断层）
Write-Host "[*] 正在静默部署 Python 3.10 开发环境..." -ForegroundColor Cyan
winget install Python.Python.3.10 --silent --accept-package-agreements

# 3. 实时刷新宿主机全局环境变量（无需重开 PS 终端，使 docker 与 python 命令立即可用）
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") + ";"
+ [System.Environment]::GetEnvironmentVariable("Path","User")
```

```
# 4. 打印并验证环境版本
Write-Host "[+] 基础设施环境并网成功!" -ForegroundColor Green
python --version
docker --version
```

第三步：宿主机 Python 核心遥测驱动与底座依赖并网

由于 TimeAudit 的遥测引擎主控 (`main.py`) 需要直接穿透至硬件与操作系统内核层，并发调用 NVIDIA 显卡内核驱动、Windows Win32 交互式窗口句柄以及 PDH 性能计数器，因此宿主机侧的 Python 环境必须具备强大的底层桥接库。

请在终端中执行以下并网依赖指令：

```
# 1. 强制升级全局包管理工具 pip，消除老旧版本带来的哈希校验异常
python -m pip install --upgrade pip

# 2. 一键安装高精硬件遥测、时空指纹指派与数据库高并发写入所需的外部驱动依赖包
# psutil：拦截系统软硬件进程状态、网卡物理流量
# pynvml：深度桥接 NVIDIA 显卡架构，读取实时功耗、显存分发、PCIe 吞吐
# asyncpg：极其强悍的 PostgreSQL 异步驱动，原生支持批量流式写入，消除吞吐瓶颈
# pywin32：访问 Windows 核心 Native API、Wintrust 数字签名链及 User32 聚焦状态机
pip install psutil pynvml asyncpg pywin32

Write-Host "[+] 宿主机高精驱动依赖库配置完毕。" -ForegroundColor Green
```

第四步：构建与引导本地时序数仓集群 (Docker Compose)

进入 TimeAudit 的项目工作目录，一键构建并激活全栈数仓。该生态包含高配 PostgreSQL 15（自动挂载本地物理卷 `./postgres_data` 进行持久化）、基于本地 `Dockerfile` 实时编译的 Ingestor 异步数据同步管道、以及无感导入了脱水纯文本配置的 Grafana 仪表盘。

```
# 1. 进驻物理工作目录
cd "E:\TimeAudit"

# 2. 引导 Docker 编排引擎，开启后台全隐形持续运行模式，并强制本地编译 Ingestor 镜像
Write-Host "[*] 正在基于 Dockerfile 本地编译并拉起数仓基础设施群..." -ForegroundColor Cyan
docker compose up -d --build

# 3. 延迟 3 秒，点检容器集群物理健康状态与端口拓扑
Start-Sleep -Seconds 3
docker compose ps

Write-Host "[+] 本地时序数仓群已就位。PostgreSQL(55432) -> Ready | Grafana(53000) -> Ready" -ForegroundColor Green
```

🔵 第五步：构建高可靠性开机自启任务（规避 Session 0 与 UAC 死锁）

⚠️ 核心架构设计缺陷修正说明

在传统的自启设计中，若将计划任务的执行主体指定为 `NT AUTHORITY\SYSTEM`，会导致任务运行在完全隔离的 **Session 0** 空间内。在 Session 0 中，Windows 交互式桌面不复存在，`context_worker.py` 中调用的 `user32.GetForegroundWindow()` 将**永远返回 NULL**，直接导致切窗状态机失效、工时数仓彻底失明。此外，若通过后台无窗口模式触发嵌套了 `RunAs` 管理员权限的批处理，会导致 UAC 确认弹窗在隐形状态下死锁挂起。

本指南提供完美的架构修正方案：**直接配置 `-LogonType Interactive` 并缺省 `-UserId` 参数**，让计划任务内核自动识别并强绑定为当前运行此脚本的活动物理用户，同时依凭 **HighestAvailable** 最高提权等级。从而在无窗口隐形守护模式下，完美闭环调用 User32 API 并获取管理员特权。

请在管理员 PS 终端中复制并执行以下全套自动化挂载脚本：

```
$TaskName = "TimeAuditTelemetryEngine_Fixed"
$TaskPath = "E:\TimeAudit\start_all.bat"

# 1. 初始化 windows 任务计划程序底层 COM 引擎
$Service = New-Object -ComObject Schedule.Service
$Service.Connect()

# 2. 获取根目录并创建一个全新的空白任务定义
$RootFolder = $Service.GetFolder("")
$TaskDefinition = $Service.NewTask(0)

# 3. 配置高级策略设置 (Settings)
$Settings = $TaskDefinition.Settings
$Settings.AllowStartIfOnBatteries = $true      # 允许笔记本在电池供电时启动
$Settings.StopIfGoingOnBatteries = $false     # 切换到电池供电时不要停止任务
$Settings.ExecutionTimeLimit = "PT0S"        # 彻底打破 3 天强制停止限制，设置为【无限期运行】

# 4. 配置触发器：绑定物理用户登录事件 (TASK_TRIGGER_LOGON = 9)
$Triggers = $TaskDefinition.Triggers
$Trigger = $Triggers.Create(9)

# 5. 配置动作：绑定引导脚本路径与工作夹 (TASK_ACTION_EXEC = 0)
$Actions = $TaskDefinition.Actions
$Action = $Actions.Create(0)
$Action.Path = $TaskPath
$Action.WorkingDirectory = "E:\TimeAudit"

# 6. 配置安全特权 (TASK_RUNLEVEL_HIGHEST = 1, TASK_LOGON_INTERACTIVE_TOKEN = 3)
$Principal = $TaskDefinition.Principal
$Principal.RunLevel = 1                      # 赋予最高可用权限 (Highest)
$Principal.LogonType = 3                     # 强绑定交互式登录令牌，完美解决 Session 0 隔离问题

# 7. 正式向 windows 内核注册并覆盖写入任务 (TASK_CREATE_OR_UPDATE = 6)
$RootFolder.RegisterTaskDefinition($TaskName, $TaskDefinition, 6, $null, $null, 3)
```

```
Write-Host "[+] 成功绕过 WMI 序列化 Bug, 通过底层 COM 引擎强行并网自启任务!" -
ForegroundColor Green
```

第六步：环境迁移、容灾快照与全量热逻辑恢复

当时序物理分区跨越长周期，或者宿主机遭遇固态硬盘损毁、系统重装时，单纯依靠冷压缩 `postgres_data` 目录可能会因为 Docker 引擎底层文件系统锁未释放而导致卷挂载损坏。以下是生产级的热逻辑快照与无损重建方案。

1. 动态容灾快照导出（热逻辑备份）

在数仓持续高并发写入的状态下，利用容器内的 `pg_dumpall` 实用工具，可以在完全不停机的情况下，将全部时序事实表（`fact_process_context`、`fact_process_activity`、`fact_system_hardware`）及关联的维表注册信息冷脱水为单文件 SQL 快照：

```
# 执行容器内全量热重构备份，将备份写入宿主机 E:\TimeAudit 目录下
Write-Host "[*] 正在执行数仓长周期时序数据冷脱水快照..." -ForegroundColor Cyan
docker exec -t audit-postgres pg_dumpall -U leyang >
E:\TimeAudit\time_audit_perfect_archive.sql
Write-Host "[+] 快照导出完毕，安全存储于 E:\TimeAudit\time_audit_perfect_archive.sql"
-ForegroundColor Green
```

2. 灾后环境秒级并网与热导入恢复

当新系统重装完毕，或者项目需要整体迁移至全新宿主机时，请直接执行以下恢复指令：

PowerShell

```
# [步骤一] 确保新环境下 E:\TimeAudit 资产已解压
cd "E:\TimeAudit"

# [步骤二] 直接一键拉起全新的、干净的 Docker 空数仓集群
Write-Host "[*] 正在重构并拉起全新空白时序数仓基础拓扑..." -ForegroundColor Cyan
docker compose up -d

# 等待数据库物理端口初始化就绪
Start-Sleep -Seconds 5

# [步骤三] 优化修正：丢弃会引发长文本假死、编码变异的 PS 管道(Get-Content | docker exec)，
# 改用 windows 原生 cmd 底层文件句柄重定向符(<)，实现大体积 SQL 数据的秒级无损灌入
Write-Host "[*] 正在注入容灾快照流，开始逻辑结构还原..." -ForegroundColor Cyan
cmd /c "docker exec -i audit-postgres psql -U leyang -d time_audit <
E:\TimeAudit\time_audit_perfect_archive.sql"

Write-Host "[+] 灾后重建完毕！数仓管线已全面恢复正常运行，历史数据实现零丢包并网。" -
ForegroundColor Green
```

补充技巧：配置 LibreHardwareMonitor 开机静默运行

为确保宿主机侧的 `hardware_worker.py` 能够持续、稳定地获取高精度的 CPU Package 功耗及核心电压数据：

1. 打开 `E:\TimeAudit\LibreHardwareMonitor.exe` 图形界面。
2. 在顶部菜单栏依序勾选: `Options` -> `Minimize On Close` (关闭时最小化)。
3. 勾选: `Options` -> `Run On Windows Startup` (随 Windows 开机自启)。
4. 勾选: `Options` -> `Minimize To Tray` (最小化到系统托盘)。

配置完成后, 该辅助驱动软件将在每次开机时以静默无干扰的状态运行在后台, 配合 `check_status_gui.ps1` 的 PID 锁安全检测与实时心跳监视, 持续为 TimeAudit 遥测管线提供高可用的 WMI 核心数据底座。